

Extending AssetOpsBench: Smart Grid Transformer Integration and Automated Scenario Generation

Sagar Chethan Kumar, Rohith Kanathur, Ananya Kapoor, Dev Bahl

Dept of Computer Science, Columbia University

New York, NY, USA

{sagar.chethankumar, rk3443, ak5447, db3791}@columbia.edu

Dhaval Patel, Kaoutar El Maghraoui

IBM T.J. Watson Research Center

Yorktown Heights, NY, USA

{pateldha, kelmaghr}@us.ibm.com

Abstract—Industrial asset operations require AI agents that can reason over heterogeneous telemetry, failure modes, maintenance records, and domain standards. Existing industrial agent benchmarks such as AssetOpsBench provide realistic multi-agent evaluation environments, but remain limited by narrow asset-class coverage and manually authored scenarios that do not scale to new industrial domains. This paper extends AssetOpsBench in two directions. First, we integrate a Smart Grid Transformer asset class with four diagnostic tools: supervised health-index prediction, dissolved-gas-analysis interpretation, winding-temperature assessment, and load-profile assessment grounded in IEC transformer standards. Second, we introduce a ScenarioGeneratorAgent pipeline for automated scenario creation. The pipeline constructs an evidence-grounded asset profile, allocates a coverage-aware scenario budget across operational domains, and generates candidate scenarios through a hybrid validation-and-repair loop that enforces schema validity, tool reachability, physical plausibility, standards alignment, and non-duplication. We profile the pipeline with PyTorch Profiler and Weights & Biases and apply two-level caching, parallel focus-group generation, thread-pool offloading, batched LLM calls, and early rejection filtering. On Smart Grid Transformer scenario generation, these optimizations reduce end-to-end runtime by up to $8\times$ for 50 scenarios while preserving quality: the optimized pipeline achieves a composite score of 74.2 ± 1.9 versus 73.8 ± 3.0 for the baseline. These results show that standards-grounded synthetic scenario generation can expand industrial agent benchmarks efficiently without sacrificing scenario validity.

I. INTRODUCTION

Industrial assets such as data-center cooling systems, power transformers, and wind turbines are complex, multi-component systems whose safe and efficient operation depends on continuous monitoring across heterogeneous data streams: time-series telemetry from IoT sensors, failure-mode libraries, maintenance work orders, and operational logs. Translating this flood of multi-modal data into timely, condition-aware decisions has historically required scarce subject-matter experts (SMEs) who must synthesise insights across data silos by hand. The emergence of large language model (LLM) based AI agents offers a qualitatively different paradigm: autonomous, goal-driven systems that can integrate perception, reasoning, and action across the full asset lifecycle, from anomaly detection and root-cause analysis to predictive maintenance planning and work-order generation.

AssetOpsBench [1] is the first unified framework designed to guide the development, orchestration, and evaluation

of domain-specific agents for Industry 4.0 asset operations. It provides a simulated industrial environment backed by a CouchDB IoT telemetry store, five specialised agents (IoT, FMSR, TSFM, WO and Vibration), and a curated set of 141 manually authored scenarios spanning single-agent utterances and multi-agent coordination tasks.

Despite this progress, two fundamental bottlenecks limit the benchmark’s scope. First, **asset-class coverage** is narrow: the initial release covers only chillers and air-handling units (AHUs), both HVAC assets with well-understood failure modes. Critical infrastructure classes, most prominently high-voltage power transformers whose health assessment relies on distinct diagnostic modalities such as dissolved gas analysis (DGA) of insulating oil, winding temperature monitoring, and electrical load profiling, are entirely absent. Agents evaluated on the existing benchmark cannot therefore be assessed on transformer diagnostic tasks, which represent some of the most economically significant maintenance operations in power utilities. Second, and more fundamentally, **scenario creation does not scale**. All 141 benchmark scenarios were manually authored by SMEs across four lifecycle phases. This process is time-consuming, difficult to standardise across asset classes, and requires deep domain knowledge that is not readily available for every new asset type. As AssetOpsBench is extended to new industrial domains, manual authoring becomes an acute bottleneck: each new asset class demands its own set of physically plausible, causally consistent, tool-reachable scenarios that comply with relevant ISO and IEC standards.

These two gaps are compounded by a shared underlying difficulty: generating high-quality evaluation scenarios for industrial agents is hard precisely because it requires satisfying simultaneous constraints that are not all of the same kind. A scenario must be *physically plausible* (respecting the laws governing the asset), *causally consistent* (observations must lead to feasible actions), *temporally consistent*, *tool-reachable* (all required MCP tools must be available with the right parameters), *standards-compliant* (aligned with ISO IEC standards), and *non-duplicative* with respect to existing scenarios. Prompt-based generation alone is insufficient because an LLM may produce linguistically fluent scenarios that violate physical or operational constraints; a separate validation and repair stage is therefore unavoidable.

This paper addresses both gaps. We first extend AssetOpsBench with a complete Smart Grid Transformer asset class with four new failure diagnostic tools: a supervised health index predictor trained on real labeled transformer data, and three LLM-based diagnostic tools grounded in IEC 60599 and IEC 60076-7. We then introduce a ScenarioGeneratorAgent pipeline that operates in three stages: asset profiling, budget allocation, and generation. The profiling stage builds a structured asset representation grounded in tool schemas, data inventories, and retrieved domain literature. The budget stage distributes the scenario count across operational domains: IoT, FMSR, TSFM, Work Order, and multi-agent; proportionally to the asset’s data modalities and tool coverage. The generation stage synthesises scenarios conditioned on the asset profile and domain policies, passing each candidate through an LLM-based repair step and deterministic validation checks before acceptance; candidates failing after bounded retries are discarded. The pipeline is instrumented with PyTorch Profiler and Weights & Biases to measure per-phase latency, token usage, and cache behaviour. Key optimizations applied include two-level caching, parallel focus-group execution, and thread pool offloading. Together, these reduce end-to-end pipeline time by up to $8\times$ over the unoptimized baseline for 50 scenarios. Scenario quality is evaluated along three dimensions: static checks, LLM judge, and dry-run agent execution, producing a composite 0–100 quality score. The optimized pipeline achieves a mean quality score of 74.2 ± 1.9 versus 73.8 ± 3.0 for the baseline, confirming that the speed gains come at no cost to scenario validity.

II. LITERATURE REVIEW

a) Industrial asset management benchmarks.: The original AssetOpsBench [1] is the closest prior work to ours. It introduces a benchmark for industrial asset operations with manually authored scenarios, specialised IoT, FMSR, TSFM, and WO agents, and an LLM-as-judge evaluation protocol. Related industrial-agent work studies language-agent access to IoT telemetry [2], CodeReAct-style work-order automation [3], and failure-mode/sensor reasoning [4]. Other industrial benchmarks, such as SustainDC [5] and WFCRL [6], evaluate control policies for data centers and wind farms. However, these efforts either rely on fixed expert-authored scenarios or focus on control and diagnostic tasks rather than scalable scenario generation for new asset classes. Our work extends AssetOpsBench by adding a new transformer asset class and replacing manual scenario authoring with a standards-grounded generation pipeline.

b) Domain-specific and application-specific agent benchmarks.: Several recent benchmarks evaluate LLM agents in specialised operational domains. ITBench [7] evaluates agents on real-world IT automation tasks spanning SRE, compliance, and FinOps, while AIOpsLab [8] constructs cloud-operations incidents from workload and fault generators. Business, workplace, and software-engineering environments such

as CRMArena-Pro [9], TheAgentCompany [10], and SWE-bench [11] similarly test long-horizon tool use in realistic digital workflows. These benchmarks provide valuable execution-grounded evaluation protocols, but they do not address the hybrid time-series, work-order, failure-mode, and standards-constrained setting of industrial asset management.

c) Generalist and multi-agent benchmarks.: Broader agent benchmarks evaluate general-purpose agents across web, OS, ML, and app-based environments. AgentBench [12], WebArena [13], WorkArena [14], OSWorld [15], and AppWorld [16] focus on interactive digital environments, while MLEBench [17], MLAGentBench [18], and MLGym [19] evaluate agents on machine-learning engineering and research workflows. These settings are useful for studying planning, tool use, and execution reliability, but they largely omit the physical-state reasoning and time-series modalities central to condition monitoring. Our benchmark extension instead targets multi-agent reasoning over industrial telemetry, diagnostic standards, maintenance evidence, and MCP-backed tools.

d) Automated and synthetic scenario generation.: The difficulty of scaling handcrafted benchmarks has motivated automated task and environment generation. TaskBench [20] constructs task-automation benchmarks from tool graphs, AgentGen [21] synthesises planning environments and tasks, and ARE [22] scales agent environments with rules, tools, and verifiers. APIGen [23], ToolLLM [24], and WizardLM [25] further show that synthetic instructions and function-calling data can improve tool-use capabilities. However, these methods target general-purpose tool use rather than industrial scenarios requiring physical plausibility, temporal consistency, standards alignment, and tool reachability. Our pipeline generates scenarios from live MCP-exposed asset evidence and validates them through a hybrid repair loop against engineering constraints, domain standards, and executable tool availability.

e) Agent architectures and evaluation methodology.: Our system builds on common agentic execution patterns, including ReAct-style reasoning and acting [26], executable code actions [27], plan-execute decomposition [28], and multi-agent planning [29]. Generalist multi-agent systems such as Magentic-One [30] motivate modular orchestration, while MCP-Universe [31] and MCP-Bench [32] evaluate agents over real MCP servers. For evaluation, we follow prior work on LLM-as-judge protocols [33] and agent-as-judge evaluation [34], while using trajectory-level failure analysis inspired by [35]. Unlike prior agent benchmarks, our evaluation setting couples trajectory quality with industrial scenario validity, standards compliance, and executable tool solvability.

f) LLM-based diagnostic tools for power transformers.: Transformer health assessment is a mature engineering discipline grounded in IEC 60599 (dissolved gas analysis and Rogers Ratio fault classification) and IEC 60076-7 (thermal loading guidelines). Prior ML work has applied neural networks to transformer fault diagnosis [36] and health index prediction [37], [38], but these models have not been integrated into an agent benchmarking framework or exposed as MCP tools. Our predict_health_index

and three LLM-based diagnostic tools (`interpret_dga`, `assess_winding_temperature`, `assess_load_profile`) bridge this gap, providing the first standards-grounded transformer diagnostic tooling within an open agent benchmark.

III. METHODOLOGY

A. Transformer Asset Integration

AssetOpsBench is extended with Smart Grid Transformers as a new asset class, covering three signal families: dissolved gas analysis (DGA) of transformer oil, winding temperature, and electrical load profiles. The IoT agent automatically discovers the transformer assets and sensor readings.

Four new MCP tools are added to the FMSR agent. `predict_health_index` uses a supervised ML model trained on a labeled transformer health dataset [37] to predict a health score from 0–100 across five condition bands. The remaining three tools are grounded in established electrical engineering standards: `interpret_dga` applies the IEC 60599 Rogers Ratio method [39], [40] to classify fault type from dissolved gas concentrations; `assess_winding_temperature` applies the IEC 60076-7 thermal model [41] to compute insulation ageing rate and thermal risk; and `assess_load_profile` derives three-phase apparent load and classifies loading status against IEC 60076-7 cyclic loading limits [41], [42]. Grounding diagnostic reasoning in IEC standards ensures that LLM outputs are verifiable against known domain criteria and that generated benchmark scenarios remain physically valid.

B. Scenario Generation Pipeline

Scaling AssetOpsBench to new asset classes remains bottlenecked by manual scenario authoring. Expert-written scenarios provide strong operational fidelity, but they are costly to produce, difficult to scale, and slow to adapt across heterogeneous assets, failure modes, sensors, and maintenance workflows.

We introduce a synthetic scenario generation pipeline that reduces this dependence on subject-matter experts while preserving grounding, diversity, and operational realism. As shown in Figure 2, the pipeline proceeds in three stages: profile, budget, and generate. It first constructs an evidence-grounded asset profile, then allocates scenarios across operational domains, and finally synthesizes and validates scenarios under domain-specific constraints.

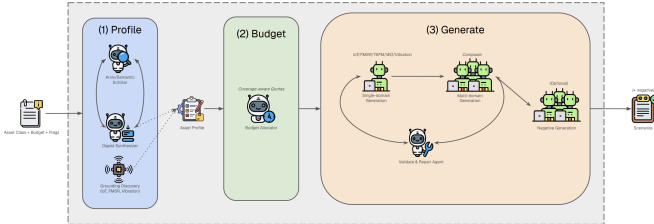


Fig. 2. Scenario Generation Pipeline. The pipeline profiles the target asset, allocates a coverage-aware scenario budget, and generates validated positive and optional negative scenarios.

1) *Profile: Grounded Asset Profiling:* The profile stage builds the asset representation used to condition generation. This profile summarizes the asset’s functional role, components, observable signals, failure modes, maintenance actions, and available tool interfaces. Rather than generating scenarios from an asset name alone, the pipeline grounds generation in available data inventories, tool schemas, and curated external evidence.

We support two regimes. In the *closed-form* regime, each scenario contains all information required for evaluation. In the *open-form* regime, scenarios reference concrete entities in the execution environment, such as asset IDs, sensor tags, time ranges, alerts, or work-order records. Open-form scenarios are generated only when these entities can be deterministically resolved.

2) *Budget: Coverage-Aware Scenario Allocation:* The budget stage determines how many scenarios to generate for each operational domain. This prevents the generator from over-representing domains with abundant surface-level information while neglecting domains that are operationally important.

The allocator uses the asset profile, available tools, data modalities, and target scenario count to distribute coverage across domains. Assets with rich telemetry may receive more monitoring and time-series tasks, while assets with detailed maintenance histories may receive more work-order or lifecycle-management tasks. Multi-agent scenarios are budgeted separately to ensure cross-domain reasoning is represented without dominating the benchmark.

3) *Generate: Scenario Synthesis and Validation:* The generation stage produces scenarios conditioned on the asset profile, domain budget, and domain-specific policies. Each scenario includes natural-language text, an operational category, a characteristic form, and grounding metadata when required.

Generation is constrained by policies that define valid tools, task types, grounding patterns, and prohibited constructions. These constraints ensure that telemetry scenarios reference valid sensors and time windows, failure-mode scenarios preserve plausible component-degradation relationships, and work-order scenarios align with maintenance-oriented actions.

Each candidate then enters a validation and repair loop. An LLM-based repair step corrects malformed or underspecified candidates, while deterministic checks enforce schema compliance, tool-domain consistency, grounding validity, duplicate removal, and multi-agent coverage requirements. Candidates that fail after bounded retries are discarded.

When multi-agent coverage is required, validated single-domain scenarios are composed into cross-domain tasks. These scenarios require agents to integrate evidence across operational views, such as linking sensor anomalies to failure modes or maintenance history. The composed scenarios are revalidated before inclusion.

The pipeline also supports optional negative scenarios. These are generated after the positive set and do not consume its budget. Negative scenarios introduce invalid identifiers, unsupported dependencies, missing time windows, or insuf-

ficient evidence to evaluate refusal behavior and uncertainty calibration.

C. Data, Model, and Optimization Scope

All stages of the `ScenarioGeneratorAgent` pipeline use `watsonx/meta-llama/llama-3-3-70b-instruct` served via IBM WatsonX as the LLM backbone. The model is invoked across all pipeline stages: asset profiling, budget allocation, and scenario generation.

Three data sources are used. Existing `AssetOpsBench` scenarios for chillers and AHUs serve as few-shot examples during generation, providing style and schema grounding. Academic literature is retrieved at runtime from ArXiv and SemanticScholar using asset-specific queries; the fetched papers are used to construct the `AssetProfile` that grounds generation in domain knowledge. The Mendeley transformer health dataset [37] provides labeled sensor readings used to train the health index regression model for the Smart Grid Transformer asset class.

Five optimizations are applied to the generation pipeline. **Two-level caching.** Asset profiles, retrieved literature, and few-shot examples are cached at two levels: a fast in-memory LRU cache and persistent disk storage. This avoids re-running expensive LLM synthesis and academic retrieval for the same asset across repeated runs.

Parallel per-domain generation. Scenario generation across focus groups (IoT, FMSR, TSFM, Work Order) runs parallelly rather than sequentially.

Thread pool offloading. Blocking LLM HTTP calls are offloaded to a shared thread pool executor, keeping the async event loop free and allowing multiple pipeline phases to make progress simultaneously without stalling on I/O.

Batch LLM calls. Multiple scenarios are requested in a single prompt rather than one call per scenario, amortizing per-call overhead, network latency, parsing, and rate-limiter tokens.

Early rejection filtering. A lightweight pre-screen eliminates clearly malformed or duplicate candidates before the full LLM validator runs, avoiding unnecessary validation cost for outputs that fail basic length, field-presence, or near-duplicate checks.

D. Profiling Tools

All per-phase times are wall-clock seconds measured via `timed_section` spans and verified against the PyTorch Profiler Chrome trace. Four profiling experiments were conducted on the Transformer asset, each isolating a single optimization dimension: scalability, caching, parallelism, and combined effect. Weights & Biases is used for run-level tracking across both the generation and evaluation stages, with all metrics accumulated during the run and flushed in a single deferred batch to avoid silent metric-drop from non-main threads.

E. Metrics Captured

Generation. Per-LLM-call metrics record wall-clock time, prompt tokens, completion tokens, and total tokens. Run-level

summary scalars capture top-level metadata, aggregate and per-phase token usage, wall-clock and CPU time per pipeline phase, cache hit/miss counts and rates, and per-focus batch aggregates including generated, valid, repaired, and rejected scenario counts with overall pass rate. Two W&B rich tables are logged: `llm_calls_table` (one row per LLM call) and `batch_results_table` (one row per batch attempt).

Evaluation. Per-scenario timeline metrics record scenario ID, type, static score (/20), LLM judge score (/30), dry-run score (/50), and composite quality score (/100). Run-level summary scalars report mean scores across all scenarios for each evaluation dimension. Table VII details the evaluation methodology.

IV. EXPERIMENTAL RESULTS

We conduct four profiling experiments to characterize the latency behavior of the scenario generator agent pipeline and to quantify the impact of the optimization techniques applied. All experiments use the Smart Grid Transformer asset in CouchDB-grounded generation mode. Times are reported in wall-clock seconds, measured using `timed_section` spans and validated against PyTorch Profiler Chrome traces.

Unless specified, all experiments use the following defaults: batch size of 10 scenarios per LLM call, 3 focus groups scenario generation in parallel, 4 threads in the I/O thread pool, a 24-hour disk cache for asset profiles, CouchDB grounded scenario generation, and ArXiv as the literature retriever. All measurements are averaged over 3 runs.

A. Experiment 1: Scalability – Pipeline Time vs. Number of Scenarios

We vary the total scenario budget $N \in \{10, 25, 50\}$ while holding all other parameters at their defaults, running the experiment independently on both the unoptimized baseline and the optimized variant. This quantifies how end-to-end latency scales with workload size and measures the absolute speedup delivered by the optimizations. Tables I and II report per-phase and total pipeline times for the baseline and the optimized variants in seconds, respectively.

TABLE I
PIPELINE TIME VS. NUMBER OF SCENARIOS – UNOPTIMIZED BASELINE.

Phase	N=10	N=25	N=50
Get Server Descriptions	2.40	2.58	2.20
Build Asset Profile	448.28	338.90	325.70
Allocate Scenario Budget	2.05	3.37	2.34
Generate & Validate Single-Agent	11.40	34.79	38.83
Generate & Validate Multi-Agent	4.05	20.67	39.32
Full Pipeline (in seconds)	468.22	400.35	408.43

In the unoptimized baseline, Phase 1 (Build Asset Profile) dominates, consuming 325–448 s and accounting for 80–96% of total pipeline time due to uncached academic retrieval, PDF fetching, and LLM synthesis on every run.

TABLE II
PIPELINE TIME VS. NUMBER OF SCENARIOS – OPTIMIZED VARIANT

Phase	N=10	N=25	N=50
Get Server Descriptions	1.56	2.42	2.16
Build Asset Profile	0.00	0.00	0.00
Allocate Scenario Budget	0.00	0.00	0.00
Generate & Validate Single-Agent	8.23	14.86	15.95
Generate & Validate Multi-Agent	10.73	17.28	18.11
Full Pipeline (in seconds)	13.63	19.88	50.86

The optimized variant eliminates this cost entirely via a two-level cache (L1 in-memory LRU + L2 disk JSON), reducing Phases 1 and 2 to zero seconds on warm runs.

The total speedup is $34\times$ at $N = 10$, $20\times$ at $N = 25$, and $8\times$ at $N = 50$ ($408\text{ s} \rightarrow 50.9\text{ s}$). The decreasing ratio at larger N reflects the growing share of scenario-generation time (Phases 3 and 4), which scales with budget. Phases 3 and 4 in the optimized variant scale roughly linearly (Phase 3: $8.2\text{ s} \rightarrow 15.9\text{ s}$ from $N = 10$ to $N = 50$) yet remain well below the baseline (38.8 s at $N = 50$), attributable to parallel focus-group execution.

B. Experiment 2: Parallelism – Pipeline Time vs. Number of Parallel Focuses

We vary $C \in \{1, 2, 3, 5\}$, which controls how many of the five focus-group generation tasks (*iot*, *fmsr*, *tsfm*, *wo*, *vibration*) execute parallel. All other parameters are fixed at $N = 50$; the asset profile and budget are served from disk cache so Phases 1 and 2 contribute near zero time.

TABLE III
PIPELINE TIME VS. NUMBER OF PARALLEL FOCUSES.

Phase	C=1	2	3	5
Get Server Descriptions	2.18	2.15	2.18	2.45
Build Asset Profile	0.00	0.00	0.00	0.00
Allocate Scenario Budget	0.00	0.00	0.00	0.00
Gen Single-Agent	43.88	22.29	17.66	17.62
Gen Multi-Agent	46.07	54.51	44.44	27.22
Full Pipeline (in seconds)	87.39	78.95	64.29	47.30

Increasing C from 1 (fully serial) to 5 reduces total pipeline time from 87.4 s to 47.3 s , a $1.85\times$ speedup. Phase 3 exhibits the clearest scaling: 43.9 s ($C=1$) $\rightarrow$ 17.6 s ($C=5$), with sharply diminishing returns beyond 3 concurrent tasks.

C. Experiment 3: Caching – Impact of Cache State on Pipeline Time

We compare two runs of the optimized variant with identical parameters ($N = 50$, all defaults) differing only in cache state. The *cold* run forces a full asset profile; the *warm* run reads the profile written by the cold run.

TABLE IV
PIPELINE TIME VS. CACHE STATE.

Phase	Cache Cold	Cache Warm
Get Server Descriptions	2.41	2.28
Build Asset Profile	247.40	0.00
Allocate Scenario Budget	0.00	0.00
Generate & Validate Single-Agent	17.30	17.23
Generate & Validate Multi-Agent	35.34	45.57
Full Pipeline (in seconds)	302.47	65.09

The disk cache eliminates Phase 1 entirely on warm runs ($247.4\text{ s} \rightarrow 0\text{ s}$), saving over four minutes per run. The full pipeline drops from 302.5 s (cold) to 65.1 s (warm): a $4.6\times$ speedup. The small increase in Phase 4 ($35.3\text{ s} \rightarrow 45.6\text{ s}$) falls within normal LLM API latency variance.

D. Experiment 4: Combined Optimizations – Baseline vs. Optimized (Cold Cache)

To fairly isolate the non-caching optimizations, we compare the unoptimized baseline against the optimized variant under identical cold-cache conditions

TABLE V
PIPELINE TIME: BASELINE VS. OPTIMIZED, BOTH COLD CACHE.

Phase	Baseline	Optimized
Get Server Descriptions	2.20	2.42
Build Asset Profile	325.70	216.85
Allocate Scenario Budget	2.34	2.56
Generate & Validate Single-Agent	38.83	14.65
Generate & Validate Multi-Agent	39.32	32.46
Full Pipeline (in seconds)	408.43	268.96

Even with the cache disabled, the optimized variant achieves a $1.52\times$ end-to-end speedup ($408.4\text{ s} \rightarrow 269.0\text{ s}$). Phase 1 improves by 33% ($325.7\text{ s} \rightarrow 216.9\text{ s}$) through I/O concurrency: the baseline executes PDF downloads, database queries, and literature searches sequentially, whereas the optimized module offloads each blocking call to a background thread pool, filling CPU-idle network-wait time with concurrent requests. Phase 3 shows the largest relative gain: a $2.65\times$ speedup ($38.8\text{ s} \rightarrow 14.7\text{ s}$) from parallel focus execution.

E. Evaluation

Optimizing the generation pipeline for speed is only meaningful if the quality of the produced scenarios is preserved. A faster pipeline that generates low-quality, unanswerable, or structurally broken scenarios would be counterproductive for benchmarking purposes. This section presents the evaluation methodology used to assess scenario quality and a comparative quality analysis between the baseline and optimized pipelines. Scenario quality is assessed along three dimensions, summarized in Table VII, summing to a 0–100 composite score. A scenario above 70 is considered high quality; below 50 indicates fundamental issues.

Dry-run execution carries the highest weight (50 pts) as it provides the strongest real-world signal: a scenario an agent can execute and answer plausibly is useful for benchmarking.

F. Quality Comparison: Baseline vs. Optimized

To verify that the optimization techniques do not degrade scenario quality, 50 Smart Grid Transformer scenarios were generated using both the unoptimized baseline and the optimized pipeline under identical conditions (CouchDB-grounded, $N = 50$), each repeated across three independent runs. Both sets were evaluated using the full three-stage methodology. Table VI reports mean $\pm$ standard deviation across the three runs for each component score and the composite quality score.

TABLE VI
SCENARIO QUALITY EVALUATION: BASELINE VS. OPTIMIZED
(MEAN $\pm$ STD, $n = 3$ RUNS).

Metric	Baseline	Optimized
Static Score (/20)	18.4 $\pm$ 0.6	19.1 $\pm$ 0.3
LLM Judge Score (/30)	22.2 $\pm$ 3.6	22.4 $\pm$ 1.8
Dry-Run Score (/50)	33.1 $\pm$ 1.3	32.7 $\pm$ 2.5
Quality Score (/100)	73.8 $\pm$ 3.0	74.2 $\pm$ 1.9

The mean quality score of the optimized pipeline (74.2 $\pm$ 1.9) is statistically indistinguishable from the baseline (73.8 $\pm$ 3.0), with a gap of only 0.4 points well within the run-to-run variance of either condition. Both pipelines exceed the 70-point high-quality threshold on average, confirming that the generated scenarios are suitable for agent benchmarking.

V. DISCUSSION

A. Interpretation

Profiling with PyTorch Profiler and W&B reveals that asset profile construction: literature retrieval, PDF fetching, and LLM synthesis accounts for 80–96% of baseline pipeline time, making it the dominant bottleneck regardless of scenario generation. Key optimizations that were applied to address these include: a two-level cache (in-memory LRU + disk JSON), a thread pool that offloads blocking I/O operations to run concurrently rather than sequentially, and parallelization that generates scenarios across focus groups simultaneously. Together these deliver a $8\times$ speedup at $N = 50$ scenarios. The quality results (74.2 vs. 73.8 mean score, optimized vs. baseline) confirm that none of these optimizations degrade scenario validity.

B. Limitations

All experiments were conducted using a single LLM backend (WatsonX Llama 3.3 70B), so it is unknown whether the speedup ratios and quality scores hold across different model families. Cross-model comparison was planned but could not be conducted due to model unavailability of closed source models; open-source alternatives would additionally require domain-specific fine-tuning to produce high quality

scenarios. Additionally, since LLM inference runs on remote IBM WatsonX infrastructure, GPU utilisation metrics captured by W&B reflect only local host activity and cannot measure true model-serving compute cost. Finally, the dry-run execution and quality judging are performed by an LLM, meaning the scores are inherently sensitive to the judge model’s own capabilities and biases.

C. Future Work

An extension to this work is cross-model comparison. Repeating the experiments across GPT-4, Llama 4, and Mistral Large would establish whether pipeline throughput is sensitive to backend latency characteristics. Broader asset class coverage including wind turbines, pumps, compressors would validate that the generation and evaluation methodology generalizes beyond transformers. On the evaluation side, replacing the LLM judge with human annotations is another future work worth looking into.

VI. CONCLUSION

We have presented two contributions that together address the dual bottleneck of asset-class coverage and scenario-authoring scalability in AssetOpsBench. First, we extended the benchmark with a complete Smart Grid Transformer asset class containing four failure mode diagnostic tools spanning a supervised health index predictor and three IEC standards-grounded LLM-based diagnostic tools (Rogers Ratio DGA interpretation, winding temperature assessment, and load profile assessment). This is, to our knowledge, the first integration of standards-grounded transformer diagnostic tooling within an open source benchmark. Second, we introduced a *ScenarioGeneratorAgent* pipeline that automates the construction of physically plausible, tool-reachable, and standards-compliant evaluation scenarios for any asset class on-boarded into the benchmark. The pipeline combines grounded asset-profile synthesis, budgeted domain allocation, focus-group generation, and a validate-and-repair loop. To the best of our knowledge, this is the first work to apply automated, knowledge-grounded scenario generation to industrial asset maintenance operations.

Our empirical study on the Smart Grid Transformer asset class demonstrates optimizations such as two-level caching, asynchronous I/O thread pooling, and parallel focus-group execution, that reduce end-to-end pipeline time by up to $8\times$ for 50 scenarios over the unoptimized baseline, profiled using PyTorch Profiler and W&B across experiments varying scenario budget, parallelism, and cache state. Crucially, these gains do not come at the cost of scenario quality: the optimized pipeline attains a composite quality score of 74.2 $\pm$ 1.9 versus 73.8 $\pm$ 3.0 for the baseline, a difference well within run-to-run variance.

REFERENCES

- [1] D. Patel, S. Lin, J. Rayfield, N. Zhou, C. Shyalika, S. R. Yarrabothula, R. Vaculin, N. Martinez, F. O’donncha, and J. Kalagnanam, “Assetopsbench: Benchmarking ai agents for task automation in industrial asset operations and maintenance,” 2026. [Online]. Available: <https://arxiv.org/abs/2506.03828>

- [2] J. T. Rayfield, S. Lin, N. Zhou, and D. C. Patel, "ReAct meets industrial IoT: Language agents for data access," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, S. Potdar, L. Rojas-Barahona, and S. Montella, Eds. Suzhou (China): Association for Computational Linguistics, Nov. 2025, pp. 364–382. [Online]. Available: <https://aclanthology.org/2025.emnlp-industry.24/>
- [3] N. Zhou, D. Patel, and A. Bhattacharyya, "Deployed ai agents for industrial asset management: Codereact framework for event analysis and work order automation," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 47, pp. 40176–40184, Mar 2026. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/41453>
- [4] C. Constantinides, D. Patel, S. Lin, C. Guerrero, S. D. Patil, and J. Kalagnanam, "Failuresensoriq: A multi-choice qa dataset for understanding sensor relationships and failure modes," 2025. [Online]. Available: <https://arxiv.org/abs/2506.03278>
- [5] A. Naug, A. Guillen, R. Luna, V. Gundecha, D. Rengarajan, S. Ghorbanpour, S. Mousavi, A. R. Babu, D. Markovikj, L. D. Kashyap, and S. Sarkar, "Sustaindc: Benchmarking for sustainable data center control," 2025. [Online]. Available: <https://arxiv.org/abs/2408.07841>
- [6] C. B. Monroc, A. Bušić, D. Dubuc, and J. Zhu, "WFCRL: A multi-agent reinforcement learning benchmark for wind farm control," 2025. [Online]. Available: <https://arxiv.org/abs/2501.13592>
- [7] S. Jha, R. Arora, Y. Watanabe, T. Yanagawa, Y. Chen, J. Clark, B. Bhavya, M. Verma, H. Kumar, H. Kitahara, N. Zheutlin, S. Takano, D. Pathak, F. George, X. Wu, B. O. Turkkan, G. Vanloo, M. Nidd, T. Dai, O. Chatterjee, P. Gupta, S. Samanta, P. Aggarwal, R. Lee, P. Murali, J. wook Ahn, D. Kar, A. Rahane, C. Fonseca, A. Paradkar, Y. Deng, P. Moogi, P. Mohapatra, N. Abe, C. Narayanaswami, T. Xu, L. R. Varshney, R. Mahindru, A. Sailer, L. Shwartz, D. Sow, N. C. M. Fuller, and R. Puri, "ITBench: Evaluating AI agents across diverse real-world IT automation tasks," 2025. [Online]. Available: <https://arxiv.org/abs/2502.05352>
- [8] Y. Chen, M. Shetty, G. Somashekar, M. Ma, Y. Simmhan, J. Mace, C. Bansal, R. Wang, and S. Rajmohan, "AIOpsLab: A holistic framework to evaluate AI agents for enabling autonomous clouds," 2025. [Online]. Available: <https://arxiv.org/abs/2501.06706>
- [9] K.-H. Huang, A. Prabhakar, O. Thorat, D. Agarwal, P. K. Choubey, Y. Mao, S. Savarese, C. Xiong, and C.-S. Wu, "CRMArena-Pro: Holistic assessment of LLM agents across diverse business scenarios and interactions," 2025. [Online]. Available: <https://arxiv.org/abs/2505.18878>
- [10] F. F. Xu, Y. Song, B. Li, Y. Tang, K. Jain, M. Bao, Z. Z. Wang, X. Zhou, Z. Guo, M. Cao, M. Yang, H. Y. Lu, A. Martin, Z. Su, L. M. Maben, R. Mehta, W. Chi, L. K. Jang, Y. Xie, S. Zhou, and G. Neubig, "TheAgentCompany: Benchmarking LLM agents on consequential real world tasks," in *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2026. [Online]. Available: <https://openreview.net/forum?id=LZnKNAPvhG>
- [11] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" 2024. [Online]. Available: <https://arxiv.org/abs/2310.06770>
- [12] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang, "AgentBench: Evaluating LLMs as agents," 2025. [Online]. Available: <https://arxiv.org/abs/2308.03688>
- [13] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig, "WebArena: A realistic web environment for building autonomous agents," 2024. [Online]. Available: <https://arxiv.org/abs/2307.13854>
- [14] A. Drouin, M. Gasse, M. Caccia, I. H. Laradji, M. D. Verme, T. Marty, L. Boisvert, M. Thakkar, Q. Cappart, D. Vazquez, N. Chapados, and A. Lacoste, "WorkArena: How capable are web agents at solving common knowledge work tasks?" 2024. [Online]. Available: <https://arxiv.org/abs/2403.07718>
- [15] T. Xie, D. Zhang, J. Chen, X. Li, S. Zhao, R. Cao, T. J. Hua, Z. Cheng, D. Shin, F. Lei, Y. Liu, Y. Xu, S. Zhou, S. Savarese, C. Xiong, V. Zhong, and T. Yu, "OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments," 2024. [Online]. Available: <https://arxiv.org/abs/2404.07972>
- [16] H. Trivedi, T. Khot, M. Hartmann, R. Manku, V. Dong, E. Li, S. Gupta, A. Sabharwal, and N. Balasubramanian, "AppWorld: A controllable world of apps and people for benchmarking interactive coding agents," 2024. [Online]. Available: <https://arxiv.org/abs/2407.18901>
- [17] J. S. Chan, N. Chowdhury, O. Jaffe, J. Aung, D. Sherburn, E. Mays, G. Starace, K. Liu, L. Maksin, T. Patwardhan, A. Madry, and L. Weng, "MLE-bench: Evaluating machine learning agents on machine learning engineering," in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=6s5uXNWGIh>
- [18] Q. Huang, J. Vora, P. Liang, and J. Leskovec, "Benchmarking large language models as AI research agents," 2024. [Online]. Available: <https://openreview.net/forum?id=N9wD4RfWY0>
- [19] D. Nathani, L. Madaan, N. Roberts, N. Bashlykov, A. Menon, V. Moens, A. Budhiraja, D. Magka, V. Vorotilov, G. Chaurasia, D. Hupkes, R. S. Cabral, T. Shavrina, J. Foerster, Y. Bachrach, W. Y. Wang, and R. Raileanu, "MLGym: A new framework and benchmark for advancing AI research agents," 2025. [Online]. Available: <https://arxiv.org/abs/2502.14499>
- [20] Y. Shen, K. Song, X. Tan, W. Zhang, K. Ren, S. Yuan, W. Lu, D. Li, and Y. Zhuang, "TaskBench: Benchmarking large language models for task automation," 2024. [Online]. Available: <https://arxiv.org/abs/2311.18760>
- [21] M. Hu, P. Zhao, C. Xu, Q. Sun, J. Lou, Q. Lin, P. Luo, and S. Rajmohan, "AgentGen: Enhancing planning abilities for large language model based agent via environment and task generation," 2025. [Online]. Available: <https://arxiv.org/abs/2408.00764>
- [22] R. Froger, P. Andrews, M. Bettini, A. Budhiraja, R. S. Cabral, V. Do, E. Garreau, J.-B. Gaya, H. Laurençon, M. Lecanu, K. Malkan, D. Mekala, P. Ménard, G. M.-T. Bertran, U. Piterbarg, M. Plekhanov, M. Rita, A. Rusakov, V. Vorotilov, M. Wang, I. Yu, A. Benhalloum, G. Mialon, and T. Scialom, "ARE: Scaling up agent environments and evaluations," 2025. [Online]. Available: <https://arxiv.org/abs/2509.17158>
- [23] Z. Liu, T. Hoang, J. Zhang, M. Zhu, T. Lan, S. Kokane, J. Tan, W. Yao, Z. Liu, Y. Feng, R. Murthy, L. Yang, S. Savarese, J. C. Niebles, H. Wang, S. Heinecke, and C. Xiong, "APIGen: Automated pipeline for generating verifiable and diverse function-calling datasets," 2024. [Online]. Available: <https://arxiv.org/abs/2406.18518>
- [24] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, L. Hong, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun, "ToolLLM: Facilitating large language models to master 16000+ real-world apis," 2023. [Online]. Available: <https://arxiv.org/abs/2307.16789>
- [25] C. Xu, Q. Sun, K. Zheng, X. Geng, P. Zhao, J. Feng, C. Tao, Q. Lin, and D. Jiang, "WizardLM: Empowering large pre-trained language models to follow complex instructions," 2025. [Online]. Available: <https://arxiv.org/abs/2304.12244>
- [26] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. R. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," in *The Eleventh International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=WE_vluYUL-X
- [27] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji, "Executable code actions elicit better LLM agents," 2024. [Online]. Available: <https://arxiv.org/abs/2402.01030>
- [28] B. Xu, Z. Peng, B. Lei, S. Mukherjee, Y. Liu, and D. Xu, "Rewoo: Decoupling reasoning from observations for efficient augmented language models," 2023. [Online]. Available: <https://arxiv.org/abs/2305.18323>
- [29] A. Li, Y. Xie, S. Li, F. Tsung, B. Ding, and Y. Li, "Agent-oriented planning in multi-agent systems," in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=EqcLAU6gyU>
- [30] A. Fournay, G. Bansal, H. Mozannar, C. Tan, E. Salinas, E. Zhu, F. Niedtner, G. Proebsting, G. Bassman, J. Gerrits, J. Alber, P. Chang, R. Loynd, R. West, V. Dibia, A. Awadallah, E. Kamar, R. Hosn, and S. Amershi, "Magentic-one: A generalist multi-agent system for solving complex tasks," 2024. [Online]. Available: <https://arxiv.org/abs/2411.04468>
- [31] Z. Luo, Z. Shen, W. Yang, Z. Zhao, P. Jwalapuram, A. Saha, D. Sahoo, S. Savarese, C. Xiong, and J. Li, "MCP-universe: Benchmarking large language models with real-world model context protocol servers," 2025. [Online]. Available: <https://arxiv.org/abs/2508.14704>
- [32] Z. Wang, Q. Chang, H. Patel, S. Biju, C.-E. Wu, Q. Liu, A. Ding, A. Rezazadeh, A. Shah, Y. Bao, and E. Siow, "MCP-bench: Benchmarking tool-using LLM agents with complex real-world tasks via MCP servers," 2025. [Online]. Available: <https://arxiv.org/abs/2508.20453>

- [33] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-bench and chatbot arena," 2023. [Online]. Available: <https://arxiv.org/abs/2306.05685>
- [34] M. Zhuge, C. Zhao, D. Ashley, W. Wang, D. Khizbullin, Y. Xiong, Z. Liu, E. Chang, R. Krishnamoorthi, Y. Tian, Y. Shi, V. Chandra, and J. Schmidhuber, "Agent-as-a-judge: Evaluate agents with agents," 2024. [Online]. Available: <https://arxiv.org/abs/2410.10934>
- [35] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica, "Why do multi-agent LLM systems fail?" in *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2026. [Online]. Available: <https://openreview.net/forum?id=fAjbYBmonr>
- [36] V. Rokani, S. D. Kaminaris, P. Karaisas, and D. Kaminaris, "Power transformer fault diagnosis using neural network optimization techniques," *Mathematics*, vol. 11, no. 22, p. 4693, 2023.
- [37] R. M. Arias Velásquez and J. V. Mejía Lara, "Root cause analysis improved with machine learning for failure analysis in power transformers," *Engineering Failure Analysis*, vol. 115, p. 104684, 2020.
- [38] B. Gorgan, P. V. Notingher, L. V. Badicu, and G. Tanasescu, "Calculation of power transformers health indexes," *Annals of the University of Craiova, Electrical Engineering Series*, vol. 34, pp. 13–18, 2010.
- [39] "Mineral oil-filled electrical equipment in service – guidance on the interpretation of dissolved and free gases analysis," International Electrotechnical Commission, Geneva, Switzerland, Standard IEC 60599, 2022.
- [40] I. B. M. Taha, S. S. M. Ghoneim, and H. G. Zaini, "Improvement of rogers four ratios and iec code methods for transformer fault diagnosis based on dissolved gas analysis," *2015 North American Power Symposium (NAPS)*, pp. 1–5, 2015. [Online]. Available: <https://api.semanticscholar.org/CorpusID:32007305>
- [41] "Power transformers – part 7: Loading guide for mineral-oil-immersed power transformers," International Electrotechnical Commission, Geneva, Switzerland, Standard IEC 60076-7, 2018.
- [42] S. Chapman, *Electric Machinery Fundamentals*, ser. Electric machinery fundamentals. McGraw-Hill Companies, Incorporated, 2005. [Online]. Available: https://books.google.com/books?id=ltg6QWnn2_4C

APPENDIX

A. Did our team use any AI tool in completing this project?

- ☐ No, we did not use any AI tool.
- ☒ Yes, we used AI assistance as described below.

B. Tool(s) Used

Claude, Cursor

C. Specific Purpose

AI assistance was used for targeted debugging and refinement tasks only. All core design decisions, experimental methodology, and reported results were produced solely by the team.

D. Sections Affected

- 1) **W&B Logging:** W&B metrics were silently dropped when `wandb.log()` was called from non-main asyncio threads during parallel focus-group generation. AI assistance was used to diagnose the root cause and restructure logging into a deferred batch flushed from the main thread after `asyncio.run()` completed.
- 2) **Thread Pool Offloading Optimization:** Thread pool offloading introduced an issue where `run_in_executor` was called with a `None` loop reference, causing `RuntimeError: cannot schedule new futures after shutdown`. AI assistance was used to identify the lifecycle mismatch and add a guard that reuses the running loop's default executor.
- 3) **Scenario Generation Prompts:** AI assistance was used to improve the LLM judge prompt and scenario generation specific prompts for clarity and reduction of ambiguous instructions that caused incorrect outputs.
- 4) **Report Writing:** Light prose refinement was used in the abstract, introduction, and discussion sections to improve grammar, clarity, and conciseness. All core ideas, methodology, analysis, and conclusions were produced entirely by the team members only.

E. How We Verified Correctness

All W&B logging fixes were verified by running the generation pipeline multiple times and confirming every expected metric (phase timings, cache hit rates, batch results, LLM call records) appeared on the W&B dashboard. The thread pool fix was verified by running the optimized pipeline many consecutive times without a `RuntimeError`. Prompt improvements were validated by re-running the full three-stage evaluation suite and confirming quality scores remained stable. All reported experiment numbers were produced by running the pipelines ourselves end-to-end. No AI-generated numbers appear anywhere in the results. Report prose changes were reviewed line-by-line by team members to ensure no false information was reported.

By submitting this project, the team confirms that the analysis, interpretations, and conclusions are our own, and that any AI assistance is fully disclosed above.

TABLE VII
EVALUATION DIMENSIONS AND SCORING CRITERIA.

Dimension	Points	Criteria
Static checks	20	Schema completeness (all required fields present) Type diversity (no single type >50% of dataset) Category alignment (maps to a recognised problem group) Text length (15–400 characters) Duplicate detection (IDs and text)
LLM judge	30	Answerability: can the task be answered? (10 pts) Tool usability: are the required tools and parameters available? (8 pts) Difficulty: not trivial, not impossibly complex (6 pts) Characteristic quality: grading rubric is specific and measurable (4 pts) Clarity: task is unambiguous to a domain expert (2 pts)
Dry-run execution	50	Plausible response: final answer addresses the task (20 pts) Correct tools used: fraction of steps using appropriate tools (15 pts) No obvious errors: fraction of steps completing without error (15 pts)
Total	100	

TABLE VIII
ROGER RATIO CODES AND FAULT DIAGNOSIS [40]

Code (R_1, R_2, R_3)	Fault Type	Code (R_1, R_2, R_3)	Fault Type
(0, 0, 0)	NF: No Fault	(1, 0, 2)	HED: High Energy Discharge
(0, 1, 0)	PDLED: PD Low Energy	(0, 0, 1)	LTH1: Thermal <150°C
(1, 1, 0)	PDHED: PD High Energy	(0, 2, 0)	LTH4: Thermal 150–300°C
(1or2, 0, 1or2)	LED: Low Energy Discharge	(0, 2, 1)	MTH: Thermal 300–700°C
Other	UD: Undetermined	(0, 2, 2)	HTH: Thermal >700°C